scienceStack

Rigorous Agentic Engineering & Theoretical Computing

A Deep Integration and Operational Manual for Google Antigravity

---

Plugin Specification & Operational Guide

Version 0.15.29 — October 2026

Core Architecture:
64 Specialized Skills
3 Dedicated Subagents
1 Unified Runtime Rule

Theoretical Domains:
PDEs, CFD, Finite Elements
Stiff ODE Kinetics, MD
Quantum Systems, Linear Algebra

Adapted from the foundational `pstack` architecture by Lauren Tan (poteto) at Cursor.
Expanded for rigorous theoretical fidelity, dimensional analysis, and numerical verification in Google Antigravity.

Contents

Executive Summary iii

1 System Overview and Core Philosophy 1
- 1.1 The Crisis of Slop in Scientific Computing . . . 1
- 1.2 The scienceStack Philosophy . . . 1
  - 1.2.1 Fearless Parallelism . . . 1
- 1.3 Architectural Heritage: From pstack to scienceStack . . . 2

2 Installation, Setup, and Model Routing 3
- 2.1 Installing the Plugin . . . 3
  - 2.1.1 Global Installation . . . 3
  - 2.1.2 Verification of Installed Components . . . 3
- 2.2 Model Configuration and Reasoning Budgets . . . 3
  - 2.2.1 The Role-Based Model Architecture . . . 3
  - 2.2.2 Interactive Setup with /setup-sciencestack . . . 4

3 The Specialized Subagent Ecosystem 5
- 3.1 Overview of Agent Roles . . . 5
- 3.2 science-master: The Scientific Implementation Delegate . . . 5
- 3.3 theory-reviewer: The Mathematical Auditor . . . 6
  - 3.3.1 The 6-Point Theoretical Audit Checklist . . . 6
- 3.4 comment-master: Eliminating Commentary Rot . . . 7

4 The Five Core Scientific Principles 8
- 4.1 Principle 1: Equation Provenance and Formal Documentation . . . 8
  - 4.1.1 Code Comparison: Bad vs. Compliant . . . 8
- 4.2 Principle 2: Dimensional Discipline & Scaling . . . 9
- 4.3 Principle 3: Preservation of Physical Conservation Laws . . . 9
- 4.4 Principle 4: Numerical Stability First . . . 10
- 4.5 Principle 5: Verification Against Analytical Limits and Golden Benchmarks . . . 10

5 Scientific Skills and Playbooks 12
- 5.1 The Master Controller: /science-mode . . . 12
- 5.2 Dedicated Scientific Skills . . . 12
  - 5.2.1 /scientific-equation-verifier . . . 12
  - 5.2.2 /numerical-stability-audit . . . 12
  - 5.2.3 /golden-dataset-benchmark . . . 13
- 5.3 Specialized Scientific Playbooks . . . 13
  - 5.3.1 Playbook: equation-implementation . . . 13
  - 5.3.2 Playbook: numerical-verification . . . 13

6 End-to-End Practical Tutorials **15**
6.1 Tutorial 1: Implementing a Stiff Kinetic System (Robertson) 15
6.1.1 Problem Statement 15
6.1.2 Executing with /science-mode 15
6.1.3 What Happens Internally 15
6.2 Tutorial 2: Conducting an Overnight Autonomous Run 16
6.2.1 The Overnight Contract Prompt 16
6.2.2 Safeguards During Autonomous Execution 16

7 Antigravity Technical Adapter Reference **17**
7.1 Runtime Tool Mapping 17
7.2 Antigravity Brain Artifacts 17

8 Complete Skill and Playbook Catalog **18**
8.1 Primary Scientific and Orchestration Skills 18
8.2 Scientific Principles Catalog 18
8.3 General Engineering and Review Skills 19

Recommended Literature & References **20**

Executive Summary

In traditional software engineering, artificial intelligence coding assistants often generate superficial code that satisfies local syntax tests but fails under boundary conditions. In **computational science and engineering**, the cost of generated "slop" is catastrophic. A flipped negative sign in a Laplacian diffusion operator turns a dissipative physical process into an exponentially explosive numerical instability. A subtle catastrophic cancellation in floating-point subtraction silently destroys the condition number of a linear solve. An unverified normalization constant renders quantum wavefunctions non-unitary.

scienceStack is an Antigravity-native agentic framework engineered specifically to eliminate ungrounded code in scientific and mathematical computing. Originating from the battle-tested `pstack` architecture created by Lauren Tan at Cursor, `scienceStack` completely re-architects the agent pairing paradigm for Google Antigravity:

1. **Theoretical Provenance**: Every formula implemented by an agent must be explicitly derived in LaTeX docstrings, cite canonical literature equations, and define coordinate metrics and sign conventions.
2. **Dimensional Discipline**: Every equation is verified for physical dimension matching ($[\text{LHS}] = [\text{RHS}]$), and transcendental arguments ($\exp, \ln, \sin$) are enforced to be dimensionless.
3. **Conservation Law Guarantees**: Numerical schemes are required to track and conserve physical invariants (energy, mass, momentum, probability norm) within specified tolerance thresholds.
4. **Numerical Stability First**: The framework strictly bans naive mathematical implementations (e.g., explicit matrix inversion $\mathbf{A}^{-1}\mathbf{b}$, unscaled floating-point differences) in favor of conditioned algorithms (`solve(A, b)`, `logsumexp`, `expm1`).
5. **Analytical Limits & Golden Datasets**: Every scientific kernel must be validated against closed-form exact solutions, the Method of Manufactured Solutions (MMS), or canonical industry benchmark datasets (e.g., Sod shock tube, Taylor-Green vortex, Robertson stiff ODEs).

With 64 automated skills, three specialized subagent personas (`science-master`, `theory-reviewer`, and `comment-master`), and native Antigravity asynchronous orchestration, `scienceStack` enables fearless parallel development while upholding the mathematical rigor demanded by modern science.

Chapter 1

System Overview and Core Philosophy

1.1 The Crisis of Slop in Scientific Computing

Modern Large Language Models (LLMs) excel at generating idiomatic code snippets, web components, and routine test scripts. However, when applied to computational physics, numerical linear algebra, biomechanics, or quantum chemistry, unassisted LLMs exhibit severe failure modes:

- **Hallucinated Equations**: Formulating plausible-looking discrete approximations that lack theoretical backing or violate fundamental symmetries.
- **Catastrophic Floating-Point Cancellation**: Evaluating expressions like $f(x) = \sqrt{x+1} - \sqrt{x}$ or $(1 - \cos x)$ for small x without using Taylor series or conditioned algebraic identities, leading to severe loss of significance.
- **Inversion of Ill-Conditioned Matrices**: Emitting $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ via `numpy.linalg.inv(A) @ b` instead of direct LU/Cholesky/QR solvers, squaring the matrix condition number $\kappa(\mathbf{A})$.
- **Violation of Noetherian Invariants**: Implementing time-integration schemes (such as standard forward Euler) for conservative Hamiltonian systems, causing artificial energy blowup over time.
- **Narrative Noise in Codebases**: Polluting clean mathematical code with conversational comments, redundant stubs, and defensive workarounds that obscure algorithmic clarity.

1.2 The scienceStack Philosophy

`scienceStack` operates on a single immutable maxim:

"If you want to go fast, go deep first."

Throughput without theoretical accuracy is counterproductive. Generating ten thousand lines of unverified simulation code produces negative value if the underlying discretization violates conservation laws.

1.2.1 Fearless Parallelism

In traditional development, delegating work to multiple concurrent background agents leads to merge conflicts, architectural drift, and silent bugs. `scienceStack` makes parallelism safe by establishing strict entry contracts, formal verification suites, and autonomous review panels:

- **Isolated Workspaces**: Concurrent subagents execute inside isolated Git branches or worktrees (`Workspace: "branch"`).
- **Theoretical Gatekeepers**: No scientific kernel is merged into the primary branch until audited by the independent `theory-reviewer` subagent.
- **Objective Golden Benchmarks**: Work is considered complete only when empirical convergence rates $O(h^p)$ or relative L_2 error thresholds against reference datasets are demonstrated.

1.3 Architectural Heritage: From pstack to scienceStack

The foundation of `scienceStack` is the `pstack` plugin developed by Lauren Tan for Cursor. `pstack` pioneered autonomous subagent orchestration, dynamic playbook matching, and strict anti-slop guidelines.

In `scienceStack_antigravity`, this framework has been completely re-engineered for the Google Antigravity runtime and specialized for scientific and mathematical computing:

Component	**Original pstack (Cursor)**	**scienceStack (Antigravity)**
Primary Orchestrator	`poteto-mode`	`science-mode`
Worker Delegate	`poteto-agent`	`science-master`
Comment Reviewer	`comment-sicko`	`comment-master`
Theoretical Auditor	*None*	`theory-reviewer`
Subagent Dispatch	Synchronous / Cursor `Task`	Asynchronous `invoke_subagent`
Decision Logging	Local TSV / commit notes	Antigravity Brain Artifacts
Scientific Playbooks	Generic feature/bugfix	`equation-implementation`, `numerical-verification`
Benchmark Suite	Generic testing	`golden-dataset-benchmark`

Table 1.1: Evolution from Cursor `pstack` to Antigravity `scienceStack`.

Chapter 2

Installation, Setup, and Model Routing

2.1 Installing the Plugin

`scienceStack` conforms to the Agent Plugins 1.0.0 Specification. It can be installed globally into Antigravity or linked directly into project workspaces.

2.1.1 Global Installation

To install `scienceStack` globally across all Antigravity sessions on your machine, execute:

```
# Validate strict compliance first
agy plugin validate /home/seo3919/AI_skills/scienceStack_antigravity
   --strict

# Install into Antigravity CLI plugin engine
agy plugin install /home/seo3919/AI_skills/scienceStack_antigravity
```

2.1.2 Verification of Installed Components

Confirm that the plugin and its agents are active:

```
agy plugin list
# Expected output:
# - sciencestack (enabled, 0.15.29)
#   Components: 64 skills, 3 agents, 1 rule

agy agent
# Expected output:
# Available agents:
# comment-master
# science-master
# theory-reviewer
```

2.2 Model Configuration and Reasoning Budgets

2.2.1 The Role-Based Model Architecture

Scientific workflows require heterogeneous cognitive capabilities. Compiling numerical kernels, writing unit tests, and verifying syntax requires fast, low-latency execution. Conversely, proving

coordinate transformation invariance, analyzing ill-conditioned eigenspaces, and architecting modular solver pipelines requires maximum reasoning depth.

`scienceStack` defines clear roles mapped to Antigravity model tiers:

Role	Default Model	Antigravity Tier	Typical Tasks
`code`	`gemini-3.8-flash-high`	`Model: "flash"`	Kernels, MMS tests, refactoring, benchmark runners
`judgment`	`claude-opus-4-6-thinking`	`Model: "pro"`	Architecture, mathematical proofs, dimensional audits
`theory-reviewer`	`claude-opus-4-6-thinking`	`Model: "pro"`	Equation verification, sign checking, invariant bounds
`comment-master`	`gemini-3.8-flash-high`	`Model: "flash"`	Dead code removal, docstring formatting
`fast-worker`	`gemini-3.8-flash-medium`	`Model: "flash_lite"`	Linting, single-fixture test loops, syntax checks

Table 2.1: Model tier mapping in Antigravity.

2.2.2 Interactive Setup with /setup-sciencestack

Run the interactive setup skill to configure model preferences and reasoning budgets:

```
/setup-sciencestack
```

The skill queries the available models in your Antigravity environment, prompts for your target reasoning budget (`small`, `medium`, `large`, `unlimited`), and persists the configuration rule to:

`~/.gemini/config/rules/pstack-models.md`

Any role set to `auto` or `inherit-parent` dynamically inherits the parent chat session's active model.

Chapter 3

The Specialized Subagent Ecosystem

3.1 Overview of Agent Roles

Antigravity executes subagents asynchronously in background worker processes. `scienceStack` provides three purpose-built subagent personas that interact to implement, audit, and clean scientific software.

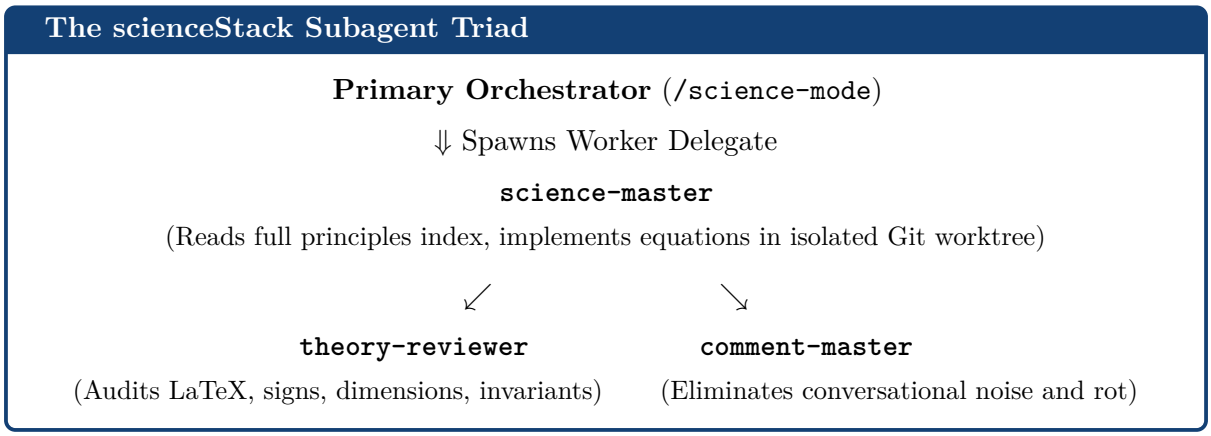


Figure 3.1: Interaction model among scienceStack subagents.

3.2 science-master: The Scientific Implementation Delegate

File: `agents/science-master.md`
Invocation: `invoke_subagent({ TypeName: "science-master", Role: "...", Prompt: "..." })`

`science-master` is the rigorous implementation engine. Whenever `science-mode` delegates a concrete coding assignment—such as implementing a finite difference stencil, constructing a sparse stiffness matrix, or writing an integration harness—it spawns `science-master`.

Antigravity Runtime Directive: science-master Operational Protocol

Before executing any file write or command, `science-master` is mandated to:

1. Read `skills/science-mode/SKILL.md` in full, including its inline Principles index.
2. Check out an isolated branch (`Workspace: "branch"`) to protect the main working directory.
3. Formulate continuous equations in LaTeX before implementing discrete algorithms.
4. Run test fixtures locally and return a consolidated diff summary to the parent orchestrator.

3.3 theory-reviewer: The Mathematical Auditor

File: `agents/theory-reviewer.md`
Invocation: `invoke_subagent({ TypeName: "theory-reviewer", Model: "pro", ... })`

`theory-reviewer` is a specialized, read-only auditor that acts as an adversarial theoretical reviewer. It never writes production code; its sole responsibility is auditing proposed diffs against governing physical laws and equations.

3.3.1 The 6-Point Theoretical Audit Checklist

When auditing a diff, `theory-reviewer` executes a rigorous verification pass:

1. **Continuous Equation Provenance**: Does every mathematical function specify its continuous formulation in LaTeX in docstrings, citing the textbook or paper equation number?
2. **Coordinate System & Sign Conventions**: Are spatial metrics (g_{ij}), coordinate systems (Cartesian, cylindrical, spherical), and sign conventions documented? For instance, in diffusion:
$$\frac{\partial u}{\partial t} = +\alpha \nabla^2 u \quad \text{(stable diffusion, } \alpha > 0\text{)} \tag{3.1}$$
A negative sign would represent reverse-time anti-diffusion, causing ill-posed exponential growth.
3. **Dimensional Homogeneity**: Do physical units match on both sides of every assignment? Are arguments to transcendental functions strictly dimensionless:
$$\exp\left(-\frac{E_a}{k_B T}\right) \quad \text{where} \quad \left[\frac{E_a}{k_B T}\right] = 1 \tag{3.2}$$
4. **Tensor and Index Order**: Are multi-dimensional arrays adhering to explicit indexing standards (row-major vs column-major, Einstein summation)?
5. **Normalization Constants**: Are integral normalizations preserved? E.g., for Fourier transforms:
$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i x \xi} dx \implies f(x) = \int_{-\infty}^{\infty} \hat{f}(\xi) e^{2\pi i x \xi} d\xi \tag{3.3}$$
Or probability densities: $\int_\Omega |\psi(\mathbf{x}, t)|^2 d\mathbf{x} \equiv 1$.
6. **Physical Invariant Conservation**: Does the discrete formulation preserve Noetherian invariants (mass, energy, momentum, symplecticity) over simulation time?

3.4 comment-master: Eliminating Commentary Rot

File: `agents/comment-master.md`
Invocation: `invoke_subagent({ TypeName: "comment-master", Model: "flash", ... })`

Comments that narrate the obvious, defend sloppy design, or preserve dead code degrade repository quality. `comment-master` conducts automated line-by-line comment sweeps:

- **Deleted**: Narrative voice ("Here we loop over nodes"), apologies ("Hack for now, fix later"), dead commented-out code blocks, obvious type restatements.
- **Preserved**: LaTeX equation docstrings, literature citations, coordinate/sign definitions, non-obvious mathematical invariants, and hardware-specific synchronization guards.

Chapter 4

The Five Core Scientific Principles

4.1 Principle 1: Equation Provenance and Formal Documentation

Skill: `skills/principle-equation-provenance/SKILL.md`

Every mathematical function or kernel in a scientific codebase must declare its provenance. Code without equation provenance is untestable against theory.

Scientific Principle: Equation Provenance

Every scientific implementation must:

1. State the continuous governing equation in LaTeX within the function docstring.
2. Cite the source paper or textbook with explicit chapter, page, or equation number.
3. Document coordinate system, metric tensor, and sign conventions.

4.1.1 Code Comparison: Bad vs. Compliant

```python
# Bad: No provenance, ambiguous variables, undocumented formula
def calc_stress(e, nu, E):
    return E / ((1 + nu) * (1 - 2*nu)) * ((1 - nu) * e[0] + nu * e[1])
```

Listing 4.1: Non-compliant ungrounded function

```python
def plane_strain_normal_stress(
    strain_xx: float,
    strain_yy: float,
    youngs_modulus: float,
    poissons_ratio: float
) -> float:
    r"""
    Compute normal stress \sigma_{xx} under isotropic linear elastic
       plane strain.

    Governing Continuous Equation:
    \begin{equation}
        \sigma_{xx} = \frac{E}{(1+\nu)(1-2\nu)} \left[
           (1-\nu)\varepsilon_{xx} + \nu\varepsilon_{yy} \right]
    \end{equation}

```

```
    Reference:
        Timoshenko, S. P., & Goodier, J. N. (1970). "Theory of
            Elasticity",
        3rd ed., McGraw-Hill, Chapter 2, Eq. (11.3), p. 34.

    Assumptions & Conventions:
        - Coordinate System: Cartesian (x, y, z)
        - Condition: Plane strain (\varepsilon_{zz} = \gamma_{xz} =
            \gamma_{yz} = 0)
        - Sign Convention: Tensile stress is positive (> 0).
    """
    E, nu = youngs_modulus, poissons_ratio
    coeff = E / ((1.0 + nu) * (1.0 - 2.0 * nu))
    return coeff * ((1.0 - nu) * strain_xx + nu * strain_yy)
```

Listing 4.2: Compliant function with Equation Provenance

4.2 Principle 2: Dimensional Discipline & Scaling

Skill: `skills/principle-dimensional-discipline/SKILL.md`

Physical quantities possess dimensions in fundamental SI units ($[M], [L], [T], [\Theta], [I]$). Dimensional mismatch is proof of a mathematical bug.

Scientific Principle: Dimensional Discipline

1. Verify dimensional homogeneity: [LHS] = [RHS] for every physical relation.
2. Nondimensionalize equations using characteristic scales prior to numerical integration:
$$x^* = \frac{x}{L_0}, \quad t^* = \frac{tU_0}{L_0}, \quad p^* = \frac{p}{\rho_0 U_0^2} \tag{4.1}$$
3. Verify that arguments to transcendental functions ($\exp, \ln, \sin, \cos$) are strictly dimensionless ($[f(\text{arg})] = 1 \implies [\text{arg}] = 1$).

4.3 Principle 3: Preservation of Physical Conservation Laws

Skill: `skills/principle-preserve-conservation-laws/SKILL.md`

According to Noether's theorem, every continuous physical symmetry corresponds to a conserved invariant:

- Time translation invariance $\implies$ Energy conservation ($\frac{dE}{dt} = 0$).
- Spatial translation invariance $\implies$ Linear momentum conservation ($\frac{d\mathbf{p}}{dt} = \mathbf{0}$).
- Spatial rotational invariance $\implies$ Angular momentum conservation ($\frac{d\mathbf{L}}{dt} = \mathbf{0}$).
- Phase invariance (gauge symmetry) $\implies$ Charge / probability norm conservation ($\int |\psi|^2 dx = 1$).

Standard dissipative discrete schemes (such as implicit backward Euler) artificially dissipate energy, while explicit schemes often artificially inject energy. `scienceStack` mandates **invariant tracking**:

$$\delta(t) = \frac{|I(t) - I(0)|}{|I(0)|} \leq \tau_{\text{drift}} \tag{4.2}$$

4.4 Principle 4: Numerical Stability First

Skill: `skills/principle-numerical-stability-first/SKILL.md`

Numerical algorithms must remain stable in floating-point arithmetic (IEEE-754). Premature vectorization must never compromise numerical conditioning.

Numerical Safety Alert: Catastrophic Cancellation

Subtracting two nearly equal floating-point numbers $a \approx b$ causes severe cancellation, wiping out significant bits:

$$\text{Error}_{\text{rel}} \approx \frac{\epsilon_{\text{mach}}}{|a - b|} \tag{4.3}$$

Use conditioned mathematical identities or special functions:

- Replace $1 - \cos x$ with $2\sin^2(x/2)$ for $|x| \ll 1$.
- Replace $\ln(1 + x)$ with `math.log1p(x)`.
- Replace $e^x - 1$ with `math.expm1(x)`.
- Replace $\ln \sum e^{x_i}$ with `scipy.special.logsumexp(x)`.

Numerical Safety Alert: Strict Matrix Inversion Prohibition

Never compute $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ via `numpy.linalg.inv(A) @ b`. Matrix inversion introduces an operation count of $2n^3$ flops and squares the numerical condition number:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\| \tag{4.4}$$

Always compute linear solves via direct factorization or Krylov iterative methods:

```
# FORBIDDEN:
x = np.linalg.inv(A) @ b

# MANDATED:
x = np.linalg.solve(A, b)
# Or for symmetric positive definite matrices:
# L = scipy.linalg.cho_factor(A); x = scipy.linalg.cho_solve(L, b)
```

4.5 Principle 5: Verification Against Analytical Limits and Golden Benchmarks

Skill: `skills/principle-verify-analytical-limits/SKILL.md`

Unit tests asserting that code "runs without throwing an exception" are insufficient for scientific software. Verification requires rigorous mathematical convergence:

1. **Analytical Closed-Form Solutions**: Validate against known exact solutions under simplified boundary conditions.
2. **Method of Manufactured Solutions (MMS)**: For nonlinear or coupled PDEs without closed-form solutions, manufacture an analytical solution $u_{\text{exact}}(\mathbf{x}, t)$, substitute into the PDE to derive an analytical source term f_{mms}, and verify that the discrete solver converges

to u_{exact} at the theoretical asymptotic rate $O(h^p)$:

$$p = \frac{\ln(E_{h_1}/E_{h_2})}{\ln(h_1/h_2)} \tag{4.5}$$

3. **Golden Datasets**: Validate against community-accepted experimental or high-resolution numerical datasets stored in `tests/golden/`.

Chapter 5

Scientific Skills and Playbooks

5.1 The Master Controller: /science-mode

`/science-mode` is the entry point for all non-trivial scientific tasks. When invoked with a user objective, it:

1. Analyzes the prompt and classifies the intent into a specialized playbook.
2. Initializes an in-context todo list or Brain Artifact.
3. Evaluates prerequisites (models, verification harnesses, git branch state).
4. Spawns subagents (`science-master`, `theory-reviewer`, `comment-master`) as each step requires.
5. Collects empirical evidence before declaring completion.

5.2 Dedicated Scientific Skills

5.2.1 /scientific-equation-verifier

Directory: `skills/scientific-equation-verifier/SKILL.md`

Performs an exhaustive audit of an existing codebase or proposed diff against mathematical and physical principles.

Usage:

```
/scientific-equation-verifier verify the stress-strain constitutive
    updates in elasticity.py
```

5.2.2 /numerical-stability-audit

Directory: `skills/numerical-stability-audit/SKILL.md`

Audits numerical algorithms for floating-point vulnerabilities:

- Matrix condition number estimates ($\kappa_2(\mathbf{A})$).
- Detects explicit matrix inversions (`np.linalg.inv`).
- Identifies unguarded divisions by zero in geometric and kinetic factors.
- Analyzes stiff ordinary differential equations (ODEs) to ensure appropriate implicit integrators (e.g., Radau IIA, BDF) are selected over explicit Runge-Kutta.

5.2.3 /golden-dataset-benchmark

Directory: `skills/golden-dataset-benchmark/SKILL.md`

Manages the acquisition, generation, and verification of canonical test problems across major scientific disciplines.

Domain	Canonical Benchmark	Validation Metric
Computational Fluid Dynamics (CFD)	1D Sod Shock Tube	L_1 and L_2 norm vs. exact Riemann solver; shock position $\Delta x < 0.5\Delta x_{\text{cell}}$
CFD / Turbulence	2D Taylor-Green Vortex	Kinetic energy decay rate $dK/dt = -2\nu\Omega$
Solid Mechanics (FEM)	Cook's Membrane	Vertical tip deflection u_y at $(48, 60)$ under mesh refinement
Chemical Kinetics	Robertson Reaction System	Mass conservation $\lvert y_1 + y_2 + y_3 - 1.0\rvert < 10^{-12}$; stiffness ratio $\approx 10^9$
Molecular Dynamics	Lennard-Jones Cluster	Microcanonical energy drift $\lvert\Delta E(t)/E_0\rvert < 10^{-5}$ over 10^5 steps
Quantum Mechanics	1D Harmonic Oscillator	Energy eigenvalue error $\lvert E_n - \hbar\omega(n + 1/2)\rvert < 10^{-8}$

Table 5.1: Canonical benchmark problems supported by /golden-dataset-benchmark.

5.3 Specialized Scientific Playbooks

Inside `skills/science-mode/playbooks/`, `scienceStack` ships specialized playbooks that structure the lifecycle of scientific implementations:

5.3.1 Playbook: equation-implementation

Path: `skills/science-mode/playbooks/equation-implementation.md`

1. **Theory Extraction**: Isolate the governing continuous differential/integral equations, boundary conditions, and sign conventions from literature.
2. **Coordinate & Tensor Settlement**: Settle metric components, array layouts, and dimensionless scaling factors.
3. **Analytical Limit Definition**: Formulate exact asymptotic solutions or MMS test fixtures.
4. **Numerical Implementation**: Spawn `science-master` to write clean, conditioned code without explicit inversions.
5. **Theoretical Audit**: Spawn `theory-reviewer` to verify signs, normalizations, and LaTeX provenance.
6. **Convergence Testing**: Prove grid convergence $O(h^p)$ against the analytical limit.

5.3.2 Playbook: numerical-verification

Path: `skills/science-mode/playbooks/numerical-verification.md`

1. **Refinement Hierarchy**: Construct grids with cell sizes $h, h/2, h/4, h/8$.
2. **Discrete Error Computation**: Measure L_2 error norm $E(h) = \|u_h - u_{\text{exact}}\|_{L_2}$.
3. **Empirical Order Calculation**: Compute convergence slope $p = \frac{\ln(E_{2h}/E_h)}{\ln 2}$.

4. **Conservation Monitoring**: Record invariant drift $\Delta I(t)$ across simulation duration.
5. **Artifact Generation**: Output convergence tables and error plots to Brain Artifacts.

Chapter 6

End-to-End Practical Tutorials

6.1 Tutorial 1: Implementing a Stiff Kinetic System (Robertson)

6.1.1 Problem Statement

The Robertson chemical reaction model represents an autocatalytic reaction with extreme timescale separation (stiffness ratio $\approx 10^9$). The governing system is:

$$\frac{dy_1}{dt} = -0.04y_1 + 10^4 y_2 y_3 \tag{6.1}$$

$$\frac{dy_2}{dt} = 0.04y_1 - 10^4 y_2 y_3 - 3 \cdot 10^7 y_2^2 \tag{6.2}$$

$$\frac{dy_3}{dt} = 3 \cdot 10^7 y_2^2 \tag{6.3}$$

with initial conditions $\mathbf{y}(0) = [1, 0, 0]^T$ over $t \in [0, 10^5]$. Total mass is strictly conserved:

$$y_1(t) + y_2(t) + y_3(t) \equiv 1.0 \quad \forall t \geq 0 \tag{6.4}$$

6.1.2 Executing with /science-mode

To instruct Antigravity to implement and verify this solver, type:

```
/science-mode implement the Robertson stiff chemical kinetics
   benchmark in kinetics.py.
Use an implicit integrator with Jacobian evaluation.
Verify that total mass y1 + y2 + y3 is conserved to within 1e-12.
Validate against reference data in tests/golden/robertson.
```

6.1.3 What Happens Internally

1. `science-mode` classifies the prompt into the `equation-implementation` playbook.
2. It verifies that `tests/golden/robertson/` exists or downloads reference trajectories.
3. It invokes `science-master` in an isolated branch. `science-master` implements the RHS and analytic Jacobian $\mathbf{J}(t, \mathbf{y})$:

$$\mathbf{J} = \begin{bmatrix} -0.04 & 10^4 y_3 & 10^4 y_2 \\ 0.04 & -10^4 y_3 - 6 \cdot 10^7 y_2 & -10^4 y_2 \\ 0 & 6 \cdot 10^7 y_2 & 0 \end{bmatrix} \tag{6.5}$$

4. `theory-reviewer` audits the Jacobian line-by-line against the partial derivatives $\partial \dot{y}_i / \partial y_j$.

5. `numerical-stability-audit` verifies that Radau IIA / BDF integration is selected and explicit RK45 is rejected.

6. Test fixtures verify that the mass conservation invariant drift remains below 10^{-12}.

6.2 Tutorial 2: Conducting an Overnight Autonomous Run

When conducting an extensive simulation study, refactoring an entire solver module, or tuning convergence across a multi-mesh grid, you can initiate an autonomous overnight run.

6.2.1 The Overnight Contract Prompt

```
/science-mode i'm stepping away. refactor the finite volume flux
   splitters in cfd/fluxes.py
to use Roe and HLLC approximate Riemann solvers.
Do not stop until:
1. All golden tests in tests/golden/sod_shock/ pass with L2 relative
   error < 0.02.
2. Invariant conservation of total mass and total energy is verified
   over 500 time steps.
3. theory-reviewer signs off on sign conventions and normalizations.
Keep a decision log in brain/artifacts so I can audit everything in
   the morning.
```

6.2.2 Safeguards During Autonomous Execution

- **Non-blocking Reactive Execution**: Antigravity does not freeze or block your shell; background tasks trigger reactive resume events.

- **Decision Trail**: The `show-me-your-work` mechanism logs every design decision, alternative rejected, and benchmark metric to a structured Antigravity Brain Artifact.

- **Self-Correction**: If a test fails with numerical instability, the agent triggers the stability audit skill (`/numerical-stability-audit`), pinpoints the flux limiter or time-step CFL violation, adjusts the formulation, and re-executes tests autonomously.

Chapter 7

Antigravity Technical Adapter Reference

7.1 Runtime Tool Mapping

The `rules/AGENTS.md` rule automatically injected by `scienceStack` instructs Antigravity how to translate Agent Plugin conventions into native Antigravity primitives:

Cursor / Standard Tool	Antigravity Equivalent	Operational Directives
`Task`	`invoke_subagent`	Background execution; sets `TypeName` and `Role`.
`AskQuestion`	`ask_question`	Multi-choice interactive prompts.
`TodoWrite`	Brain Artifacts	Persistent Markdown state in `<appDataDir>/brain/<id>/`.
`Bash`	`run_command`	Shell execution (`rg`, `find`, `pytest`).
`Read / Edit`	`read_file / edit_file`	Exact character-sequence replacement tools.
`environment: "cloud"`	`Workspace: "branch"`	Clones/branches workspace for clean isolation.
`environment: "local"`	`Workspace: "inherit"`	Shares root repository directly.

Table 7.1: Adapter tool mapping in Google Antigravity.

7.2 Antigravity Brain Artifacts

All audit trails, convergence tables, and execution summaries generated by `scienceStack` are written to the Antigravity Artifact directory:

`<appDataDir>/brain/<conversation-id>/`

These artifacts support GitHub-style alerts (`NOTE`, `TIP`, `IMPORTANT`, `WARNING`, `CAUTION`), interactive Mermaid flowcharts, and embedded convergence plots.

Chapter 8

Complete Skill and Playbook Catalog

8.1 Primary Scientific and Orchestration Skills

Skill / Slash Command	Description & Purpose
`/science-mode`	Primary orchestrator. Matches user prompt to playbooks, manages subagents, and drives tasks to empirical verification.
`/setup-sciencestack`	Configures model choices and reasoning budgets per role; generates `pstack-models.md`.
`/science-help`	Interactive guide. Recommends skills and drafts prompts for any coding situation.
`/scientific-equation-verifier`	6-point theoretical audit for LaTeX provenance, signs, dimensions, and normalizations.
`/numerical-stability-audit`	Deep inspection for condition numbers, catastrophic cancellation, zero division, and stiff ODEs.
`/golden-dataset-benchmark`	Catalog of canonical test problems (CFD, FEM, MD, quantum) and reference dataset acquisition.

8.2 Scientific Principles Catalog

Principle Skill	Rule & Core Mandate
`principle-equation-provenance`	Continuous equation in LaTeX, literature citation, coordinate/sign documentation.
`principle-dimensional-discipline`	Validate $[LHS] = [RHS]$, non-dimensionalization, dimensionless transcendental inputs.
`principle-preserve-conservation-laws`	Explicitly monitor and conserve energy, mass, momentum, and norm invariants.
`principle-numerical-stability-first`	Eliminate cancellation; ban `inv(A) @ b`; use conditioned linear solvers.
`principle-verify-analytical-limits`	Method of Manufactured Solutions (MMS), closed-form benchmarks, $O(h^p)$ convergence.
`principle-laziness-protocol`	Bias toward deletion and the smallest change that solves the problem.
`principle-foundational-thinking`	Get data structures and core types right before writing algorithmic logic.
`principle-redesign-from-first-principles`	Redesign as if the requirement were foundational from day one.

Principle Skill	Rule & Core Mandate
`principle-attack-the-premise`	When multiple fixes fail, write down and test the shared underlying premise.
`principle-subtract-before-you-add`	Remove dead code and redundant wrappers before building new features.
`principle-minimize-reader-load`	Collapse one-caller wrappers and shrink mutable state scope.

8.3 General Engineering and Review Skills

Skill / Slash Command	Description & Purpose
`/architect`	Settle caller usage, interfaces, types, and module shape before implementing code.
`/arena`	Run N parallel subagent attempts at the same design problem, then synthesize the best components.
`/swarm`	Deploy parallel subagents across slices, packages, or test suites, returning a single aggregated report.
`/interrogate`	Multi-model adversarial review of proposed diffs against safety, performance, and style standards.
`/tdd`	Test-driven bug reproduction: write a failing fixture first, verify failure, then implement fix.
`/no-comments`	Strip narrative comments before review; spawns `comment-master`.
`/unslop`	Clean conversational filler, apologies, and AI markers from documentation and code.
`/show-me-your-work`	Record reviewable decision trails to TSV files and Antigravity Brain Artifacts.
`/how`	Deep walkthrough of how an existing system or subsystem operates.
`/why`	Investigates architectural rationale using git history, issue records, and chat context.
`/teach`	Synthesizes `how` and `why` into a structured diagrammatic explanation.
`/ketchup`	Catch-up summary of chat changes since your last message with actionable decisions.
`/blast-radius`	Analyze downstream breakage risk outside the diff, proving safety with running tests.
`/benchmark-checklist`	Vets performance measurements (timers, warmups, repeat runs) before reporting numbers.

Recommended Literature & References

1. **Numerical Methods & Linear Algebra**:
   - Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
   - Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press.
   - Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM.
2. **Partial Differential Equations & Verification**:
   - Roache, P. J. (2009). *Fundamentals of Verification and Validation*. Hermosa Publishers.
   - LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press.
   - Salari, K., & Knupp, P. (2000). *Code Verification by the Method of Manufactured Solutions*. Sandia National Laboratories Report SAND2000-1444.
3. **Ordinary Differential Equations & Chemical Kinetics**:
   - Hairer, E., & Wanner, G. (1996). *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2nd ed. Springer.
   - Robertson, H. H. (1966). "The numerical integration of some systems of stiff ordinary differential equations." In *Numerical Analysis: An Introduction*, pp. 178–182.